

Local Client Verbindung und systemd-Service

1. Ziel der Aufgabe

Ziel dieser Aufgabe war es, den lokalen Console-Client mit der auf der EC2-Instanz laufenden FastAPI-Anwendung zu verbinden und die Anwendung anschließend als dauerhaften Hintergrunddienst einzurichten.

Dabei sollte geprüft werden, ob der lokale Client über die öffentliche EC2-Adresse mit dem Server kommunizieren kann. Zusätzlich sollte sichergestellt werden, dass die Anwendung auch nach dem Schließen der SSM-Session weiterläuft.

2. Ausgangssituation

Die EC2-Instanz war bereits vorbereitet und über AWS Systems Manager Session Manager erreichbar. Das GitHub-Repository lag auf der EC2-Instanz unter folgendem Pfad:

```
/opt/social-media-app
```

Die FastAPI-Anwendung befand sich nicht direkt in einer main.py im Projektstamm, sondern im Unterordner web_app. Dadurch musste beim Starten von Uvicorn der korrekte Modulpfad verwendet werden.

3. Repository und Umgebungsdateien aktualisieren

3.1 Aktuellen Stand auf der EC2 laden

Auf der EC2-Instanz wurde der aktuelle Stand des main-Branches aus GitHub gezogen:

```
cd /opt/social-media-app
git pull origin main
```

Damit wurde sichergestellt, dass der Server mit dem aktuellen Projektstand arbeitet.

3.2 Aktuellen Stand lokal laden

Auch auf dem lokalen Rechner wurde der aktuelle Stand aus dem GitHub-Repository geladen, damit Client und Server-Code zusammenpassen:

```
git pull origin main
```

3.3 .env.example zu .env kopieren

Die vorhandene Datei .env.example dient als Vorlage. Daraus wurde lokal eine echte .env-Datei erstellt. Diese Datei enthält die konkrete Konfiguration für die lokale Umgebung.

```
cp .env.example .env
```

In der lokalen .env wurde die SERVER_URL so gesetzt, dass der Console-Client auf die öffentliche EC2-Adresse mit Port 8000 zeigt:

```
SERVER_URL=http://<EC2_PUBLIC_IP>:8000
```

Die Datei .env.example bleibt als Vorlage im Repository erhalten. Die .env enthält dagegen konkrete lokale Werte und wird üblicherweise nicht ins Repository gepusht.

4. FastAPI-Server auf EC2 starten

4.1 Richtigen Uvicorn-Startbefehl ermitteln

Beim ersten Startversuch trat ein Importfehler auf, weil Uvicorn zunächst mit dem falschen Modulpfad gestartet wurde. Die Suche nach der Startdatei ergab, dass die relevante main.py im Ordner web_app liegt.

```
find . -name "main.py"
```

Relevantes Ergebnis:

```
./web_app/main.py
```

Daraus ergab sich folgender korrekter Startbefehl:

```
python -m uvicorn web_app.main:app --host 0.0.0.0 --port 8000
```

Der Parameter --host 0.0.0.0 ist notwendig, damit die Anwendung nicht nur intern auf der EC2-Instanz, sondern auch von außen über die öffentliche EC2-Adresse erreichbar ist.

4.2 Datenbank- und Rechteproblem beheben

Beim Starten der Anwendung trat zunächst ein SQLite-Fehler auf:

```
sqlite3.OperationalError: unable to open database file
```

Die Datenbank-URL war in der .env wie folgt gesetzt:

```
DATABASE_URL=sqlite:///./database.db
```

Das bedeutet, dass die Datei database.db direkt im Projektordner /opt/social-media-app erstellt werden soll. Der Benutzer ssm-user hatte jedoch keine Schreibrechte im Projektordner. Dadurch konnte SQLite die Datenbankdatei nicht anlegen.

Das Problem wurde gelöst, indem der Besitz des Projektordners auf ssm-user gesetzt wurde:

```
sudo chown -R ssm-user:ssm-user /opt/social-media-app
```

Danach konnte die Anwendung erfolgreich starten und die SQLite-Datenbankdatei erstellen.

5. EC2 Security Group für Port 8000

Damit der lokale Client den Server erreichen kann, musste HTTP-Traffic auf Port 8000 in der Security Group der EC2-Instanz erlaubt werden.

Einstellung	Wert
Typ	Custom TCP
Port	8000
Quelle	Eigene IP bzw. für Testzwecke passende Freigabe
Zweck	Zugriff auf die FastAPI-Anwendung vom lokalen Client

SSH blieb weiterhin deaktiviert. Der administrative Zugriff erfolgt weiterhin über AWS Systems Manager Session Manager.

6. API-Erreichbarkeit testen

Nach dem Start der Anwendung wurde die FastAPI-Dokumentation im Browser über die öffentliche EC2-Adresse getestet:

```
http://<EC2_PUBLIC_IP>:8000/docs
```

Die Swagger-Oberfläche war erreichbar. Damit wurde bestätigt, dass die FastAPI-Anwendung läuft, Port 8000 erreichbar ist und der Server von außen angesprochen werden kann.

7. Lokalen Console-Client testen

7.1 Python-Version lokal korrigieren

Beim Start des lokalen Clients trat zunächst ein SyntaxError auf. Ursache war, dass lokal Python 3.9 verwendet wurde, der Client-Code jedoch match/case nutzt. Dieses Sprachfeature ist erst ab Python 3.10 verfügbar.

Zusätzlich verlangten installierte Abhängigkeiten ebenfalls eine neuere Python-Version. Deshalb wurde lokal eine virtuelle Umgebung mit Python 3.12 verwendet.

```
python3.12 -m venv .venv
source .venv/bin/activate
python --version
python -m pip install -r requirements.txt
```

Danach konnte der lokale Console-Client gestartet werden:

```
python -m console_app.client
```

7.2 Verbindung zum EC2-Server prüfen

Im lokalen Client wurde die Funktion zum Anzeigen der Benutzer getestet. Es waren zwar noch keine Benutzer vorhanden, es trat jedoch kein Verbindungsfehler auf. Dadurch wurde bestätigt, dass der lokale Client grundsätzlich mit der API auf der EC2-Instanz kommunizieren kann. Im Client-Code wurde anschließend festgestellt, dass der lokale Client den falschen Endpunkt aufruft:

8. Anwendung als systemd-Service einrichten

8.1 Problemstellung

Beim manuellen Start von Uvicorn im Terminal läuft die Anwendung nur so lange, wie die SSM-Session aktiv ist. Wird die Session geschlossen, kann der Prozess beendet werden. Für einen dauerhaften Betrieb ist das ungeeignet.

Deshalb wurde die Anwendung als systemd-Service eingerichtet. Dadurch läuft sie als Hintergrunddienst unabhängig von der aktiven Terminalsitzung.

8.2 systemd-Service-Datei erstellen

Für die Anwendung wurde eine systemd-Service-Datei angelegt:

```
sudo nano /etc/systemd/system/social-media-app.service
```

Die Service-Datei startet die Anwendung aus dem Projektverzeichnis heraus mit der Python-Umgebung des Projekts und bindet Uvicorn an Port 8000.

```
[Unit]
Description=Social Media FastAPI App
After=network.target

[Service]
User=ssm-user
Group=ssm-user
```

```
WorkingDirectory=/opt/social-media-app
EnvironmentFile=/opt/social-media-app/.env
ExecStart=/opt/social-media-app/.venv/bin/python -m uvicorn web_app.main:app --host 0.0.0.0
--port 8000
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

8.3 Service aktivieren und prüfen

Nach dem Erstellen der Service-Datei wurde systemd neu geladen und der Service gestartet:

```
sudo systemctl daemon-reload
sudo systemctl start social-media-app
sudo systemctl status social-media-app
```

Zusätzlich wurde der automatische Start nach einem Neustart der Instanz aktiviert:

```
sudo systemctl enable social-media-app
```

Damit kann die Anwendung über systemctl verwaltet werden:

```
sudo systemctl start social-media-app
sudo systemctl stop social-media-app
sudo systemctl restart social-media-app
sudo systemctl status social-media-app
```

8.4 Persistenz nach Logout testen

Nach dem Einrichten des systemd-Services wurde die SSM-Session geschlossen. Anschließend wurde die Webseite erneut im Browser geladen.

Die FastAPI-Dokumentation blieb weiterhin erreichbar. Damit wurde bestätigt, dass die Anwendung nicht mehr an die aktive SSM-Session gebunden ist, sondern dauerhaft als Hintergrunddienst läuft.

9. Ergebnis

Die lokale Verbindung zum EC2-Server wurde erfolgreich getestet. Die FastAPI-Anwendung ist über die öffentliche EC2-Adresse auf Port 8000 erreichbar und der lokale Console-Client kann grundsätzlich mit dem Server kommunizieren.

Zusätzlich wurde die Anwendung als systemd-Service eingerichtet. Dadurch läuft sie nach dem Schließen der SSM-Session weiter und kann über systemctl verwaltet werden. Der Service wurde außerdem für den automatischen Start nach einem Neustart vorbereitet.